Reflection on Group Assignment 2IRR00: The SilverSonic Project

Maciej Małek
Eindhoven University of Technology
2294192, Group 24

1 Project Description and Scope

The primary objective of this project was to develop, an accessible audiobook player designed specifically for elderly users. To overcome common usability barriers, the system features a simplified interface with clearly labeled controls and three color themes to choose from. The application environment is fully active. Our current implementation focuses on core functionalities, including playback management, speed adjustments, and automatic progress saving.

2 Project Reflection

The project resulted in a fully functional software implementation that successfully meets our core technical milestones. However, the development revealed substantial friction within our team collaboration and internal workflow management.

2.1 Process and Collaboration

Managing the team dynamics required strong leadership due to significant alignment challenges. Following critical feedback on our first submission, I independently restructured and rewrote it to meet the standards. During the implementation phase, I developed the player overview frontend and backend, alongside the backend logic for home overview timestamps. My own features were completed successfully and were easy to integrate without issues.

In contrast, implementing some of the other features took an excessive amount of time. In the final phase of the project, I had to assume a leadership role and act as the main reviewer for all pull requests. This became a very tedious task because some team members completely ignored our internal deadlines. Furthermore, they possessed limited knowledge of Git and object-oriented programming in Java. I fully understood that they might have less experience than me or other two members, and I hoped that they would show the initiative to learn and deliver solid work. However, it often appeared to me that they made no effort to look up information online or in other sources, which resulted in the use of an entirely different structural template for one feature branch and caused issues with version control. I had to dedicate a large amount of unallocated time to helping them fix their personal features. I provided extensive comments on GitHub, but teammates sometimes did not even read them or apply my fixes, which caused frustration and hindered our progress further. Fortunately, during the final step of the project, another teammate helped me significantly to resolve these outstanding issues and successfully finalize the implementation.

2.2 Testing and Quality Assurance

Maintaining strict quality assurance was essential to ensuring application stability through a collaborative testing strategy. I worked closely with a fellow teammate to co-author comprehensive test cases. Additionally, every pull request required at least one formal approval before it could be merged into the main branch. Changes highlighted during these reviews had to be fully implemented before approval was granted. On a technical level, we executed rigorous black-box unit testing using the JUnit 5 framework to ensure the software met user expectations reliably.

2.3 Design Principles and Patterns

The architecture incorporates standard object-oriented design patterns to ensure modularity. For global configuration, the application employs the Singleton pattern within the Settings class to maintain a single instance of user preferences. Audio playback behavior is decoupled using the Strategy pattern, where the PlaybackController interface defines the contract and AudioPlayer provides the concrete JavaFX implementation. Data persistence avoids code duplication through the Template Method pattern via an abstract PropertiesStore extended by ProgressStore and TimestampStore. Finally, the system conforms strictly to the Model-View-Controller (MVC) architecture across all views.

Despite these structural strengths, one system deficiencies exists. We had to abandon the planned font-size adjustment feature due to development constraints.

3 Self-Reflection

An analysis of my personal contributions highlights a strong ownership of quality, and the high workload directly accelerated my technical growth. The initial brainstorming was a team effort, but the burden of refining our core models and documentation after early failures fell entirely on me. I had to carry the project through its critical phases to ensure it can be delivered on schedule. This responsibility was demanding, but it forced me to rapidly master advanced features of Java and solve complex version control conflicts in Git.

4 Conclusion and Learnings

In conclusion, the development demonstrated the necessity of architectural discipline and active team management. Navigating a project of this scale showed that software success is determined as much by communication as it is by writing code. The collaboration barriers and technical disparities presented continuous challenges, but taking on a decisive leadership role allowed us to deliver the project. Ultimately, the experience has reshaped my perspective, teaching me that managing human friction is an inseparable part of software engineering.